Data analysis and biostatistics

Jordi Vivas Sakuta and Izan Flores Tortosa

Exercises for practice 1

All the exercises for this practice will be written in a single R markdown script file (.Rmd extension). Use comments in the code to explain what you are trying to achieve within each section. Please, keep each exercise statement in your R script as a comment in order to make the code more readable.

The exercises in practice 1 will be carried out using the R built-in dataset named "iris". This dataset contains the measurements in centimeters of the variables sepal length, sepal width, petal length and petal width, respectively, for 50 flowers from each of 3 species of iris. The species are setosa, versicolor, and virginica.

The first step before starting the exercise consist in loading the iris dataset using the following command:

```
data(iris)
```

Then, data is accessed using the iris variable. For example, the View() function can be used to visualize the complete data table.

1- Basic operations with variables and vectors

Calculate the sums of all the numeric columns of the iris dataset (Sepal.Length, Sepal.Width, Petal.Length and Petal.Width). Each sum results must be stored in a new variable. Then use the print() function to display the results in the R Console. Check out the sum() function to easily perform the sums.

Tip, you can combine print() and paste() functions to display multiple elements combined in the R console, for example:

```
print(paste("I'm printing multiple elements. Here it is a number:", 42,
            " and the Sepal.Length of the first observation:", iris$Sepal.Length[1]))
```

```
#Write your code here
#muestra del sumatorio de cada elemento del dataframe
suma_data1<-sum(iris$Sepal.Length[])
suma_data2<-sum(iris$Sepal.Width[])
suma_data3<-sum(iris$Petal.Length[])
suma_data4<-sum(iris$Petal.Width[])
print(paste("Sepal.Length:",suma_data1,"Sepal.Width:",suma_data2,"Petal.Length:",suma_data3,"Petal.Widt
```

```
## [1] "Sepal.Length: 876.5 Sepal.Width: 458.6 Petal.Length: 563.7 Petal.Width: 179.9"
```

2- Subsetting

- Create a new data.frame which contains only the first ten rows of iris data. Store the result in a variable (Tip: you can use 1:10 to access the first ten rows).

Tip, you can use the knitr::kable() function to render a data.frame to a formatted table in the exported PDF:

```
knitr::kable(iris)
```

```
#Write your code here
#render del nuevo data frame que contiene las primeras 10 filas del data frame original
iris2<-iris[1:10, ]
knitr::kable(iris2)
```

Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
5.1	3.5	1.4	0.2	setosa
4.9	3.0	1.4	0.2	setosa
4.7	3.2	1.3	0.2	setosa
4.6	3.1	1.5	0.2	setosa
5.0	3.6	1.4	0.2	setosa
5.4	3.9	1.7	0.4	setosa
4.6	3.4	1.4	0.3	setosa
5.0	3.4	1.5	0.2	setosa
4.4	2.9	1.4	0.2	setosa
4.9	3.1	1.5	0.1	setosa

- Create a new data.frame which contains only the observations of the specie "versicolor".

Tip, explore the == operator to select the rows corresponding to the "versicolor" specie. In example:

```
iris$Species == "versicolor"
```

```
#Write your code here
#render del dataframe que contiene la especie versicolor
df_versicolor<-iris[iris$Species == "versicolor", ]
knitr::kable(df_versicolor)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
51	7.0	3.2	4.7	1.4	versicolor
52	6.4	3.2	4.5	1.5	versicolor
53	6.9	3.1	4.9	1.5	versicolor
54	5.5	2.3	4.0	1.3	versicolor
55	6.5	2.8	4.6	1.5	versicolor
56	5.7	2.8	4.5	1.3	versicolor
57	6.3	3.3	4.7	1.6	versicolor
58	4.9	2.4	3.3	1.0	versicolor
59	6.6	2.9	4.6	1.3	versicolor
60	5.2	2.7	3.9	1.4	versicolor

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
61	5.0	2.0	3.5	1.0	versicolor
62	5.9	3.0	4.2	1.5	versicolor
63	6.0	2.2	4.0	1.0	versicolor
64	6.1	2.9	4.7	1.4	versicolor
65	5.6	2.9	3.6	1.3	versicolor
66	6.7	3.1	4.4	1.4	versicolor
67	5.6	3.0	4.5	1.5	versicolor
68	5.8	2.7	4.1	1.0	versicolor
69	6.2	2.2	4.5	1.5	versicolor
70	5.6	2.5	3.9	1.1	versicolor
71	5.9	3.2	4.8	1.8	versicolor
72	6.1	2.8	4.0	1.3	versicolor
73	6.3	2.5	4.9	1.5	versicolor
74	6.1	2.8	4.7	1.2	versicolor
75	6.4	2.9	4.3	1.3	versicolor
76	6.6	3.0	4.4	1.4	versicolor
77	6.8	2.8	4.8	1.4	versicolor
78	6.7	3.0	5.0	1.7	versicolor
79	6.0	2.9	4.5	1.5	versicolor
80	5.7	2.6	3.5	1.0	versicolor
81	5.5	2.4	3.8	1.1	versicolor
82	5.5	2.4	3.7	1.0	versicolor
83	5.8	2.7	3.9	1.2	versicolor
84	6.0	2.7	5.1	1.6	versicolor
85	5.4	3.0	4.5	1.5	versicolor
86	6.0	3.4	4.5	1.6	versicolor
87	6.7	3.1	4.7	1.5	versicolor
88	6.3	2.3	4.4	1.3	versicolor
89	5.6	3.0	4.1	1.3	versicolor
90	5.5	2.5	4.0	1.3	versicolor
91	5.5	2.6	4.4	1.2	versicolor
92	6.1	3.0	4.6	1.4	versicolor
93	5.8	2.6	4.0	1.2	versicolor
94	5.0	2.3	3.3	1.0	versicolor
95	5.6	2.7	4.2	1.3	versicolor
96	5.7	3.0	4.2	1.2	versicolor
97	5.7	2.9	4.2	1.3	versicolor
98	6.2	2.9	4.3	1.3	versicolor
99	5.1	2.5	3.0	1.1	versicolor
100	5.7	2.8	4.1	1.3	versicolor

3- Flow control

- Write a program using loops and conditionals to calculate the maximum value of each iris column. Store the maximum of each column in vector. Store the row index of each maximum value in another vector. Do not use the max() function or any apply() function since we are practicing with the flow control.

Tip, explore the following R functions to simplify your life: rep(), ncol() and nrow().

```r
#Write your code here
#busca el maximo valor de cada columna y se assigna en un vector
maximo<-c(0,0,0,0)
for(i in 1:nrow(iris)){
 for(j in 1:(ncol(iris)-1)){
  if(maximo[j]<iris[i,j])
   maximo[j]<-iris[i,j]
 }
  }
```

- Write a program which counts how many observations exists of the "versicolor" specie. You must only use flow control and basic Boolean operations.

```r
#Write your code here
#Contador de la cantidad de iris de la especie versicolor que hay
obs <- 0
iris3 <- iris[iris$Species == "versicolor", ]
 for(i in 1:(nrow(iris3))){
  obs <- obs + 1
  }
```

4- Functions

- Write a function to calculate the average value of a numeric vector. Using the R built in function mean() is not allowed in this exercise, use a loop instead.

Tip, you can use the length() function to extract the number of elements in a vector.

```r
#Write your code here
mitja<-function(columna){
suma<-0
for(i in 1:nrow(iris)){

   suma<-suma+columna[i]
   media <- suma/length(columna)

}
return(media)
}
```

- Use the previous function to calculate the average values of each variable in the iris dataset. Use a loop to execute your average() function on each data.frame column.

```r
#Write your code here
mi<-c(0,0,0,0)
for(i in 1:(ncol(iris)-1)){
 dt<-iris[,i]
 mi[i]<-mitja(dt)
}
```

- Take a look at the following function to extract all values from a given column name (variable) and a selected specie:

```
getValuesBySpecieVar <- function( dataset, specie, variable)
{
  selRows <- which(dataset$Specie == specie)
  selCol <- which(colnames(dataset) == variable)
  dataset[selRows, selCol]
}
```

So, when this function is executed with the following arguments:

- dataset = iris
- specie = "setosa"
- variable = "sepal.length"

The result is: 5.1, 4.9, 4.7, 4.6, 5.0, 5.4, 4.6, . . . (values from sepal.length column in setosa specie)

Answer the following questions:

- What is the purpose of colnames() function and how is it used here? -En este caso el colnames mira cada nombre de las columnas de dataset para ver si se da la condicion de que sea igual a el nombre variable
- Looking at which() function documentation and its usage in the given getValuesBySpecieVar() function explain how does it work.
- especifica los valores de la condicion determinada en este caso para las filas la especie y para las columnas el nombre de la columna

5- Working with list data structures

Use the previously function: *getValuesBySpecieVar()* to create a list object named sepalLengthList which contains the data for only for the sepal length. Use the following structure:

sepalLengthList

- "setosa": 5.1, 4.9, 4.7, .
- "versicolor": 7.0, 6.4, 6.9, .
- "virginica": 6.3, 5.8, 7.1, .

To do so, use a loop that iterates for every possible specie in the iris dataset (Tip: you can use the R unique() function to select each specie only once). Then, use the *getValuesBySpecieVar()* function with the variable parameter set to "Sepal.Length" to obtain the values in each loop iteration. Finally, use the [[]] operator to assign the values to a new item in the list.

```
#Write your code here
sepalLengthList<-list()
specie<-unique(iris$Species)
for(i in specie){
sepalLengthList[[i]]<-getValuesBySpecieVar(iris,i,"Sepal.Length")
}
print(sepalLengthList)
```

```
## $setosa
##  [1] 5.1 4.9 4.7 4.6 5.0 5.4 4.6 5.0 4.4 4.9 5.4 4.8 4.8 4.3 5.8 5.7 5.4 5.1 5.7
## [20] 5.1 5.4 5.1 4.6 5.1 4.8 5.0 5.0 5.2 5.2 4.7 4.8 5.4 5.2 5.5 4.9 5.0 5.5 4.9
## [39] 4.4 5.1 5.0 4.5 4.4 5.0 5.1 4.8 5.1 4.6 5.3 5.0
##
## $versicolor
##  [1] 7.0 6.4 6.9 5.5 6.5 5.7 6.3 4.9 6.6 5.2 5.0 5.9 6.0 6.1 5.6 6.7 5.6 5.8 6.2
## [20] 5.6 5.9 6.1 6.3 6.1 6.4 6.6 6.8 6.7 6.0 5.7 5.5 5.5 5.8 6.0 5.4 6.0 6.7 6.3
## [39] 5.6 5.5 5.5 6.1 5.8 5.0 5.6 5.7 5.7 6.2 5.1 5.7
##
## $virginica
##  [1] 6.3 5.8 7.1 6.3 6.5 7.6 4.9 7.3 6.7 7.2 6.5 6.4 6.8 5.7 5.8 6.4 6.5 7.7 7.7
## [20] 6.0 6.9 5.6 7.7 6.3 6.7 7.2 6.2 6.1 6.4 7.2 7.4 7.9 6.4 6.3 6.1 7.7 6.3 6.4
## [39] 6.0 6.9 6.7 6.9 5.8 6.8 6.7 6.7 6.3 6.5 6.2 5.9
```

6- Using apply functions

- Use the apply() function combined with your average() function to calculate the average of each variable in the iris dataset.

```
#Write your code here
apply(iris[,1:4],2,mitja)
```

```
## Sepal.Length  Sepal.Width Petal.Length  Petal.Width
##     5.843333     3.057333     3.758000     1.199333
```

- Validate your average results by doing the same replacing your average() function with the R's built-in mean() function.

```
#Write your code here
apply(iris[,1:4],2,mean)
```

```
## Sepal.Length  Sepal.Width Petal.Length  Petal.Width
##     5.843333     3.057333     3.758000     1.199333
```

- Execute the following code chuck that uses an apply() function combined with subsetting and explain how does it work. What is the purpose of the - sign before the which(colnames(iris) == "Species"))?

```
versicolorAvg <- apply(iris[iris$Species=="versicolor",-which(colnames(iris) == "Species")], 2, mean)
print(versicolorAvg)
```

```
## Sepal.Length  Sepal.Width Petal.Length  Petal.Width
##        5.936        2.770        4.260        1.326
```

- Use the previously created "sepalLengthList" List and the lapply() function to calculate the average of each sepal length in each specie from the list data structure.

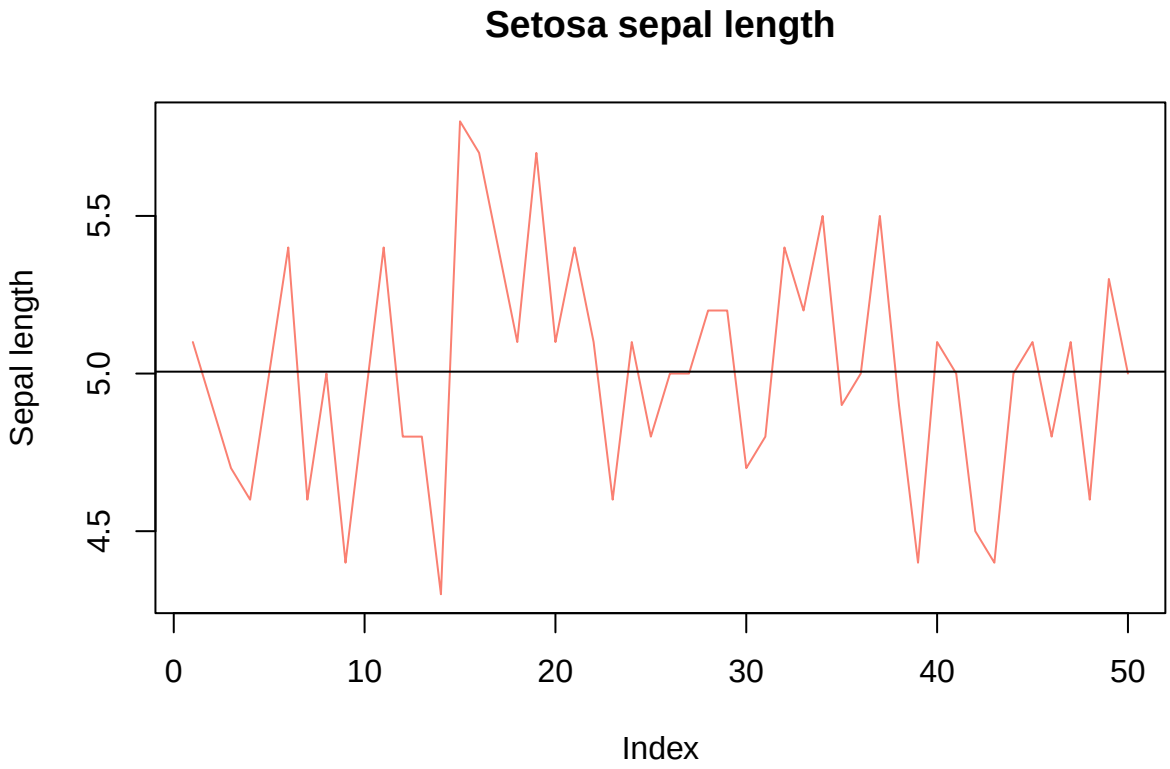
```
#Write your code here
lapply(sepalLengthList,quantile, probs = 1:3/4)
```

```
## $setosa
## 25% 50% 75%
## 4.8 5.0 5.2
##
## $versicolor
## 25% 50% 75%
## 5.6 5.9 6.3
##
## $virginica
##   25%   50%   75%
## 6.225 6.500 6.900
```

7- Plotting data with R

- Plot the Sepal.Length values only for the setosa specie using lines instead of points. Overlay the average of plotted data using the abline() function. The resulting plot should look similar to the example plot 1 (see the "1-Example_plots.pdf" file).

```
#Write your code here
iris4 <- iris[iris$Species == "setosa","Sepal.Length" ]
mplot <- mean(iris4)
plot(iris4,type="l",col="salmon", ylab="Sepal length")
title("Setosa sepal length")
abline(h=mplot)
```



- Plot the Petal.Length versus Petal.width for all species coloring the data points according its own specie. Use the legend() function to create a legend to easily identify the color corresponding to each specie. The resulting plot should look similar to the example plot 2 (see the "Example_plots.pdf" file).

```
#Write your code here
iris5 <- iris[,"Petal.Length" ]
iris6 <- iris[,"Petal.Width" ]
color<-c("setosa"="black","versicolor"="red","virginica"="green")
plot(iris5,iris6, col=color[iris$Species], xlab="Petal length", ylab= "Petal width", pch=16)
title("Petal length vs. width")
species <- c("setosa", "versicolor", "virginica")
legend("topleft", legend=species, pch=16, col=color)
```

